

Programação Aplicada

Interfaces Gráficos

Contentores e Gestores de Posicionamento

Marco Veloso

marco.veloso@estgoh.ipc.pt

(Revisto e adaptado a partir do original de António José Mendes e Maria José Marcelino)

Elementos Gráficos, AWT e Applets

Interface Gráficos

Componentes de um Interface Gráfico

Gestão de Eventos

Contentores e Gestores de Posicionamento

Interacção com o utilizador

Caixas de diálogo e menus

Elementos Multimédia

Listagens e Tabelas

Formatações gráficas

Contentores e Gestores de Posicionamento

Interface gráfico - Exemplo

Interfaces Gráficas – Contentores e Gestores de Posicionamento

Exemplo completo:

```
public class Botao extends JFrame {  
    // Construtor  
    public Botao() {  
        // 1 obtém contentor principal  
        Container contentor = getContentPane();  
        // 2 define a disposição  
        contentor.setLayout ( new FlowLayout() );  
  
        // 3a cria elemento gráfico  
        JBUTTON b1 = new JBUTTON("Botao Um");  
        // 3b adiciona elemento gráfico ao contentor principal  
        contentor.add(b1);  
  
        //...  
    }  
    //...  
}
```

Um **contentor** (*container*) é um tipo especial de componente que **serve para agrupar outros componentes** (eventualmente outro contentor)

Por exemplo, um **JPanel** é um contentor e, por isso, podem-lhe ser adicionados outros componentes

A cada contentor é adicionado um **gestor de posicionamento** (*layout manager*) que controla a forma como os seus componentes são mostrados

Cada contentor tem dois métodos:

- **setLayout (...)** - Indica qual o **gestor de posicionamento** a usar
- **add (...)** - **Adiciona** um componente ao contentor

Alguns contentores (`JPanel` e `JApplet`) têm que ser **ligados a uma superfície gráfica** (e.g. uma *applet* é ligada a um *browser* ou a um *applet viewer*), não sendo autónomos

Outros contentores (`JWindow`, `JFrame`, `JDialog`) podem ser **movidos independentemente**, são autónomos, recebendo como argumento o texto a apresentar no friso superior

```
JFrame frame = new JFrame("Janela Principal");
```

A utilização de contentores implica que **os componentes sejam primeiro adicionados ao contentor, sendo este depois adicionado à frame** (janela principal, usualmente uma implementação `JFrame`)

Assim, uma interface gráfica é conseguida pela **junção de contentores e outros componentes**

O desenho da interface gráfica é uma parte fundamental do desenvolvimento de aplicações - não esquecer que **para um utilizador comum a interface é o sistema**

Contentores não autónomos:

Classe	Função
JPanel	é o contentor mais simples, associado a outros contentores. Um JPanel é um espaço para colocar outros componentes, incluindo outros painéis
JApplet	contentor que necessita de um <i>Applet Viewer</i> para ser executado

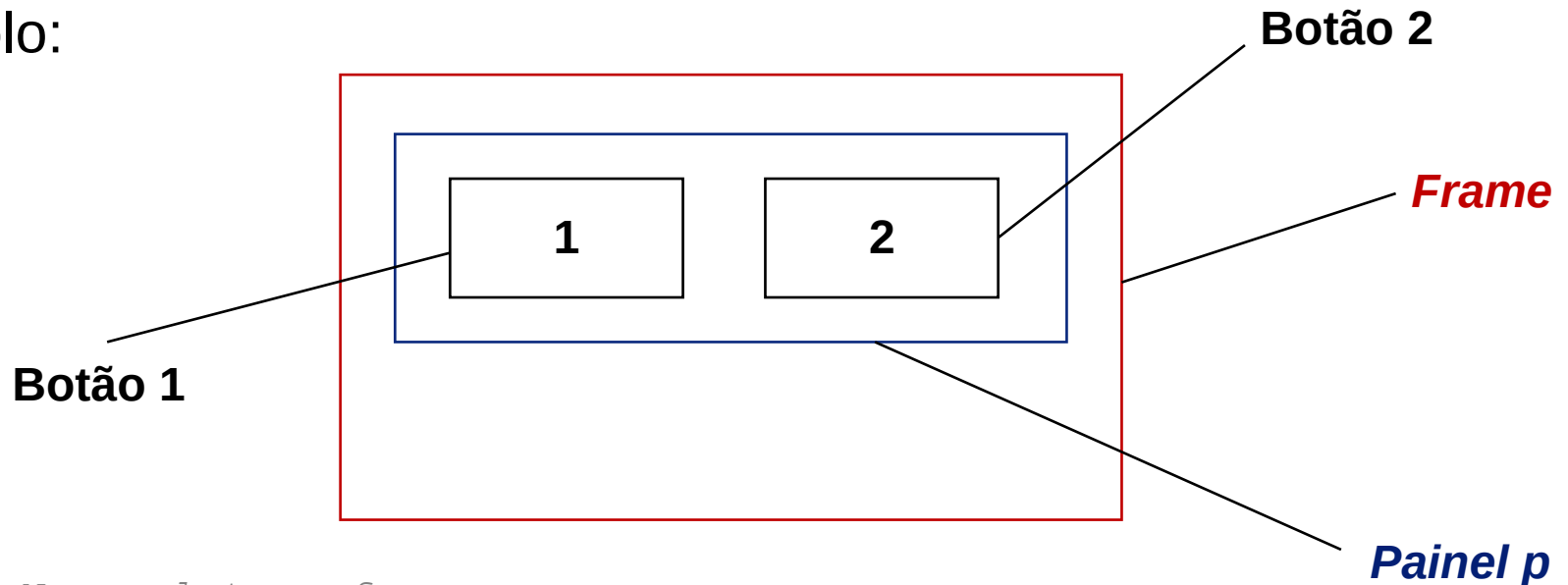
Contentores autónomos:

Classe	Função
JWindow	é uma janela autónoma ‘ <i>top-level</i> ’ sem ‘ <i>borders</i> ’ e sem menu
JFrame	é uma janela autónoma ‘ <i>top-level</i> ’ com um título e um ‘ <i>border</i> ’
JDialog	é uma janela autónoma com o propósito de apresentar mensagens (informação, erro, etc.) ao utilizador

Aplicação de elementos num contentor

Interfaces Gráficas – Contentores e Gestores de Posicionamento

Exemplo:



```
// Na applet ou frame
JButton botao1 = new JButton ("1");
JButton botao2 = new JButton ("2");
JPanel p = new JPanel();
p.add (botao1);
p.add (botao2);
Container cont = getContentPane();
cont.add (p);
```

Elementos gráficos

Interfaces Gráficas – Contentores e Gestores de Posicionamento

Os elementos gráficos (**componentes: JComponent**) mais comuns numa interface gráfica são os seguintes :

- Etiquetas (**JLabel**);
- Campos de texto (**TextField** e **PasswordField**);
- Áreas de texto (**TextArea**);
- *Combo boxes* (**ComboBox**);
- Botões (**Button** e **Choice**);
- Botões de selecção (**CheckBox**);
- Grupos de botões de rádio (**RadioButton** e **ButtonGroup**);
- Listas (**List**);
- Tabelas (**Table**);
- Potenciómetros (**Scrollbar** e **Slider**) .

Nota: todas as componentes são subclasses (herdam) de **JComponent**

<http://download.oracle.com/javase/tutorial/uiswing/components/index.html>

<http://java.sun.com/docs/books/tutorial/uiswing/components/index.html>

<http://java.sun.com/docs/books/tutorial/uiswing/examples/components/index.html>

Gestores de Posicionamento

Interfaces Gráficas – Contentores e Gestores de Posicionamento

Um **gestor de posicionamento** (*layout manager*) é um objecto que decide como distribuir os componentes num contentor

Cada contentor tem um gestor de posicionamento por defeito (as `JDialogs` e `JFrames` possuem como gestor de posicionamento por defeito o **`BorderLayout`** enquanto as `Applet` e `Panel` possuem como gestor de posicionamento por defeito o **`FlowLayout`**)

Há seis gestores de posicionamento pré-definidos:

- **`FlowLayout`**
- **`BorderLayout`**
- **`GridLayout`**
- **`GridBagLayout`**
- **`CardLayout`**
- **`BoxLayout`**

Gestores de Posicionamento

Interfaces Gráficas – Contentores e Gestores de Posicionamento

<i>Layout</i>	<i>Função</i>
FlowLayout	<i>'Layout' de defeito para JPanels</i> ; pressupõe que os objectos sejam colocados de forma centrada, da esquerda para a direita, sequencialmente e em linhas sucessivas.
BorderLayout	<i>'Layout' de defeito para todos os 'Content Panes' (JDialogs e JFrames)</i> ; divide o contentor em cinco áreas: <i>'North', 'South', 'West', 'East' e 'Center'</i> ; o tamanho relativo das várias áreas depende do tamanho dos objectos colocados em cada uma; o programador especifica em qual das áreas pretende cada componente.
GridLayout	objectos são colocados numa grelha com um determinado número de linhas e colunas, ocupando cada um uma célula da grelha; todas as células têm a mesma dimensão.
GridBagLayout	grelha bidimensional com células de tamanhos diferentes em que os componentes podem ocupar várias linhas ou colunas.
CardLayout	objectos são colocados de forma sobreposta, mas só um está visível em cada instante; pode-se controlar qual o componente visível.
BoxLayout	posiciona os objectos numa única linha ou coluna. As dimensões de cada objecto são respeitadas, e é possível definir qual o alinhamento dos objectos.

Para **alterar o tipo de gestor de posicionamento** pode ser utilizado o **método** `setLayout(...)`

Se, por exemplo, se colocar a instrução

```
setLayout ( new GridLayout(2,2) )
```

no construtor do contentor, o seu gestor de posicionamento passará a ser o *GridLayout* (grelha com células de igual tamanho) com **2** linhas por **2** colunas.

Alternativamente **é possível posicionar os objectos em localizações fixas** (*'layout'* absoluto).

Para isso, o *layout* é definido como *null* e depois de se adicionar o objecto ao *'content pane'* usa-se o **método `setBounds()`** para posicionar o objecto na localização pretendida

não é aconselhado para aplicações que pretendam ser independentes da plataforma

Gestores de Posicionamento

Interfaces Gráficas – Contentores e Gestores de Posicionamento

FlowLayout (<http://java.sun.com/docs/books/tutorial/uiswing/layout/flow.html>)

- Por defeito para *Applet* e *Panel*
- Os componentes são **centrados**
- Os componentes são colocados da esquerda para a direita, passando para a linha seguinte quando já não couber na linha corrente
- É possível alterar o alinhamento através das propriedades: **LEFT**, **CENTER** e **RIGHT**:

```
Jpanel myPanel = new JPanel( new FlowLayout( FlowLayout.LEFT ) )
```



Gestores de Posicionamento

Interfaces Gráficas – Contentores e Gestores de Posicionamento

BorderLayout (<http://java.sun.com/docs/books/tutorial/uiswing/layout/border.html>)

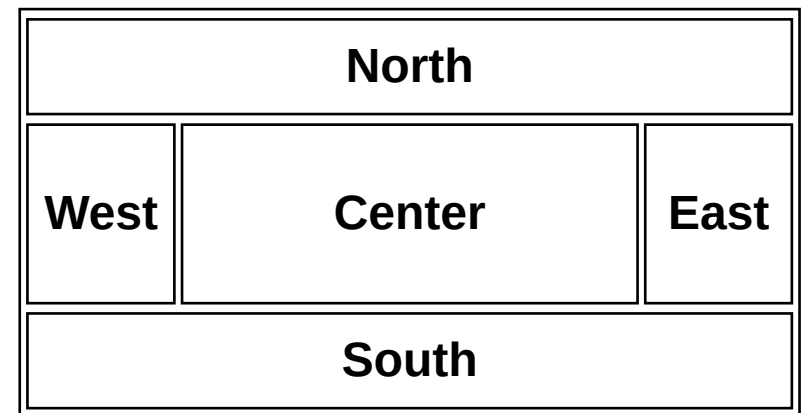
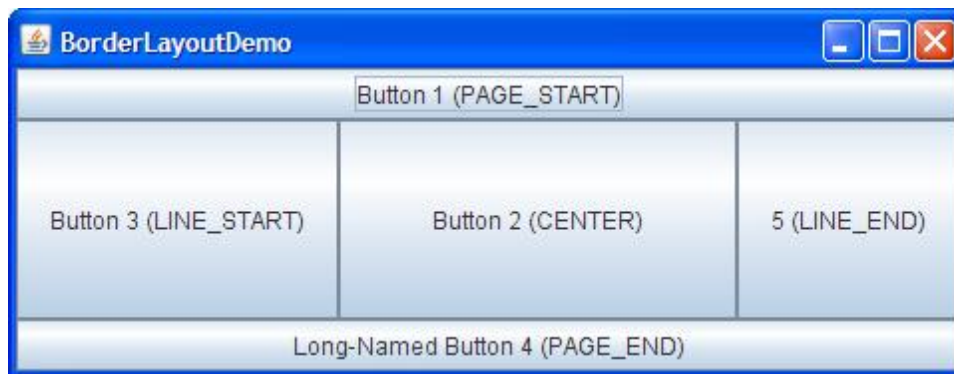
- Tem **cinco locais para colocar componentes**:

North, South, West, East e Center;

- Por defeito para todos os ‘*Content Panes*’ (*JDialogs* e *JFrames*);
- O programador **especifica em qual das áreas** quer cada componente

e.g.: `contentor.add(myPanel, BorderLayout.NORTH);`

- O **tamanho relativo das áreas** é definido pelo tamanho dos componentes que lá são colocados

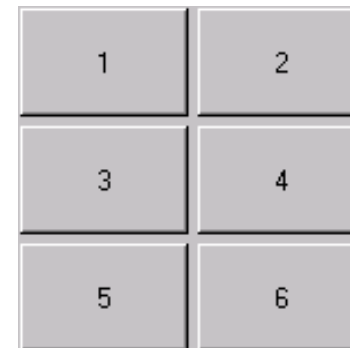


Gestores de Posicionamento

Interfaces Gráficas – Contentores e Gestores de Posicionamento

GridLayout (<http://java.sun.com/docs/books/tutorial/uiswing/layout/grid.html>)

- Os **componentes são colocados numa grelha** com um determinado número de linhas e colunas
- Ao adicionar cada componente é possível especificar a posição pretendida, caso contrário vai ser disposto na primeira célula livre
- Cada componente é colocado numa célula
- **Todas as células têm a mesma dimensão**
- É possível especificar o **espaçamento horizontal e vertical** entre as células, através dos métodos **setHgap()** e **setVgap()**

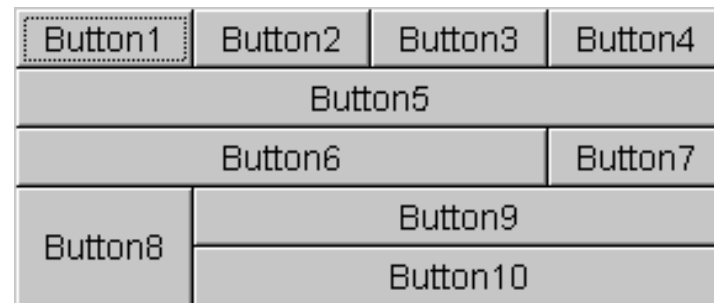
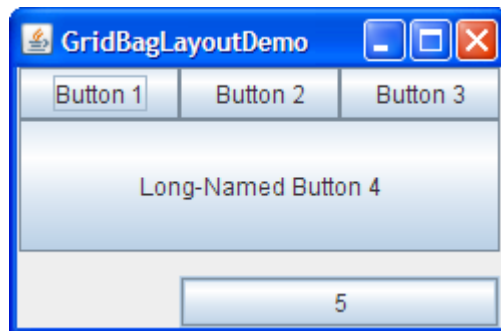


Gestores de Posicionamento

Interfaces Gráficas – Contentores e Gestores de Posicionamento

GridBagLayout (<http://java.sun.com/docs/books/tutorial/uiswing/layout/gridbag.html>)

- Uma **grelha bidimensional** de linhas e colunas
- **Nem todas as células são do mesmo tamanho**
- Os **componentes podem ocupar várias linhas ou colunas**
- Cada componente é associado com um conjunto de restrições

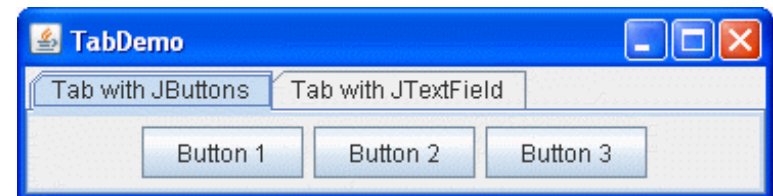
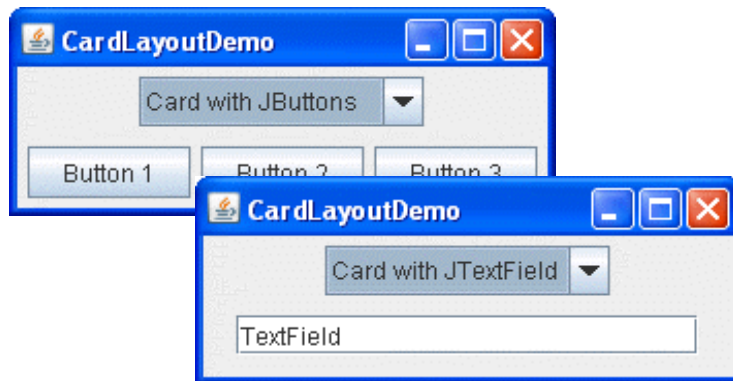


Gestores de Posicionamento

Interfaces Gráficas – Contentores e Gestores de Posicionamento

CardLayout (<http://java.sun.com/docs/books/tutorial/uiswing/layout/card.html>)

- Os **componentes são colocados uns em cima dos outros**
- **Apenas um componente está visível em cada momento**
- Os métodos podem controlar qual dos componentes é colocado visível (através do método **setVisible()** ou **show()**)

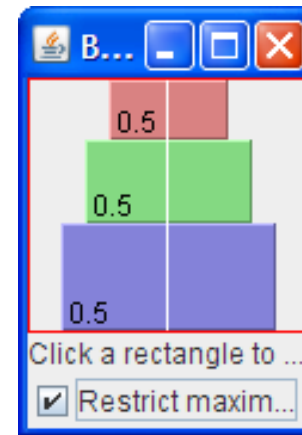
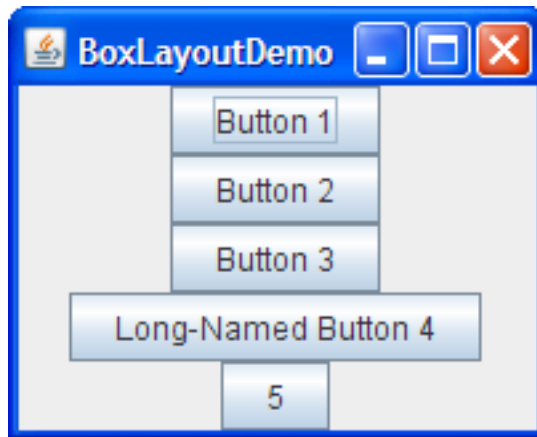


Gestores de Posicionamento

Interfaces Gráficas – Contentores e Gestores de Posicionamento

BoxLayout (<http://java.sun.com/docs/books/tutorial/uiswing/layout/box.html>)

- **Posiciona os objectos numa única linha** (variável **X_AXIS**) **ou coluna** (variável **Y_AXIS**) (podem ainda ser definida a disposição por **LINE_AXIS** OU **PAGE_AXIS**)
- As dimensões de cada objecto são respeitadas, e **é possível definir qual o alinhamento dos objectos**



Gestores de Posicionamento - Exemplo

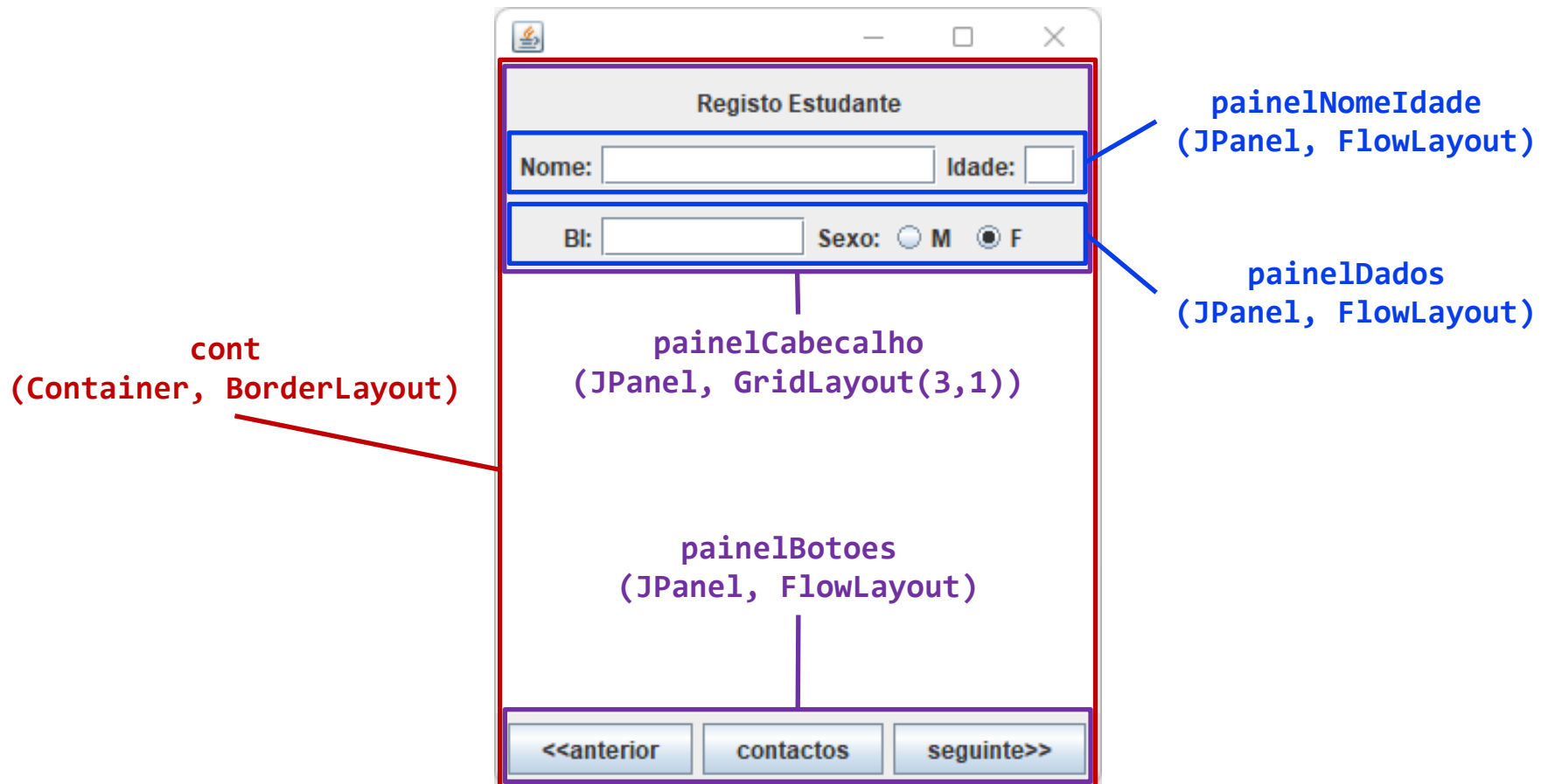
Interfaces Gráficas – Contentores e Gestores de Posicionamento



The image shows a Java Swing window titled "Registo Estudante". The window has a standard title bar with a minimize button, a maximize button, and a close button. The main content area is divided into two sections. The top section has a light gray background and contains three input fields: "Nome:" followed by a text box, "Idade:" followed by a text box, and "BI:" followed by a text box. To the right of the "BI:" field is a "Sexo:" label followed by two radio buttons, "M" and "F", with the "F" button selected. The bottom section of the window has a white background and contains three buttons: "<<anterior", "contactos", and "seguinte>>".

Gestores de Posicionamento - Exemplo

Interfaces Gráficas – Contentores e Gestores de Posicionamento



Gestores de Posicionamento - Exemplo

Interfaces Gráficas – Contentores e Gestores de Posicionamento

```
import java.awt.*;
import javax.swing.*;

public class Formulario extends JFrame
{
    public Formulario()
    {
        //painel com os campos nome e idade
        JPanel painelNomeIdade =
            new JPanel(new FlowLayout());

        painelNomeIdade.add(
            new JLabel("Nome:", SwingConstants.RIGHT));
        JTextField textoNome = new JTextField(15);
        painelNomeIdade.add(textoNome);

        painelNomeIdade.add(
            new JLabel("Idade:", SwingConstants.RIGHT));
        JTextField textoIdade = new JTextField(2);
        painelNomeIdade.add(textoIdade);

        //painel com os campos BI e sexo
        JPanel painelDados = new JPanel(new FlowLayout());

        painelDados.add(
            new JLabel("BI:", SwingConstants.RIGHT));
        JTextField textoBI = new JTextField(9);
        painelDados.add(textoBI);

        JRadioButton masc = new JRadioButton("M", false);
        JRadioButton fem = new JRadioButton("F", true);
        ButtonGroup Sexo = new ButtonGroup();
        sexo.add(masc);
        sexo.add(fem);

        painelDados.add(new JLabel("Sexo:"));
        painelDados.add(masc);
        painelDados.add(fem);

        //comp. de nomeIdade, dados e titulo,
        //grelha de 3 lin e 1 col.
        JPanel painelCabecalho =
            new JPanel(new GridLayout(3,1));
        painelCabecalho.add(new JLabel("Registo Estudante",
            (int)CENTER_ALIGNMENT));
        painelCabecalho.add(painelNomeIdade);
        painelCabecalho.add(painelDados);

        //painel com os três botões
        JPanel painelBotoes = new JPanel(new FlowLayout());
        JButton botaoAnterior = new JButton("<<anterior");
        painelBotoes.add(botaoAnterior);
        JButton botaoContactos = new JButton("contactos");
        painelBotoes.add(botaoContactos);
        JButton botaoSeguinte = new JButton("seguinte>>");
        painelBotoes.add(botaoSeguinte);

        //colocação do painéis no ContentPane da applet
        Container cont = getContentPane(); // 1
        cont.setLayout(new BorderLayout()); // 2
        cont.add(painelCabecalho, "North"); // 3
        cont.add(new JTextArea(), "Center");
        cont.add(painelBotoes, "South");
    }
}
```



Em resumo, para **construir a interface gráfica**:

- (1) **Declarar** os **componentes** e os seus *listeners*
- (2) **Instanciar** e inicializar componentes e *listeners*
- (3) **Adicionar** os componentes aos seus contentores (se houver)
- (4) **Adicionar** os componentes (e/ou contentores) à *frame*
- (5) **Registar** os *listeners* nos respectivos componentes

Para **tratar eventos**:

- Escrever métodos na *frame* para cada tipo de acção que o utilizador possa efectuar
- Escrever uma classe (ou classes) para cada tipo de *listener*, incluindo os métodos respectivos

Transição de painéis

Se uma interface for desenhada para apresentar **diferentes componentes numa área comum** (`JPanel`, de acordo com as acções do utilizador), sempre que houver necessidade de alterar o *layout* da área, deve-se:

- 1 – remover as componentes anteriores através do método `remove()`;
- 2 – adicionar as novas componentes através do métodos `add()` (2.1 se necessário activar a visibilidade da componente através do método `setVisible(true)`);
- 3 – invocar o método `revalidate()`;
- 4 – invocar o método `repaint()`;

O método **repaint()** é invocado sempre que uma componente **necessita de se “redesenhar”**, usualmente em situações de limpeza (*cleanup*) de ecrã (usualmente é invocado automaticamente quando o *focus* ou a dimensão da janela são alterados)

O método **revalidate()** (equivale à conjunção dos métodos `invalidate()` e `validate()`) deve ser **invocado sempre que componentes sejam removidos, ou novos componentes adicionados**

Esta invocação **reinicializa a listagem de componentes activos**, sendo uma indicação para uma limpeza (*repaint*) das regiões consideradas ‘sujas’ (*dirty regions*), ou seja, que sofreram alterações

Em alternativa ao método `revalidate()` também pode ser invocado o método **updateUI()**

A área de transição deverá ser constituída por um **painel** (`JPanel`) **principal**

A forma mais eficiente será criar novos painéis com os vários *layouts* pretendidos, que serão os **painéis auxiliares**

Sempre que houver uma transição de *layout*, **removem-se todos os painéis auxiliares do painel principal, e adiciona-se o painel auxiliar pretendido** (que deve ser visível)

Só no final deste procedimento se invocam os métodos `repaint()` e `revalidate()`

Transição de painéis

Interfaces Gráficas – Contentores e Gestores de Posicionamento

```
public class MyClass extends JFrame implements ActionListener {
    JPanel painelPrincipal, painelAuxiliar1, painelAuxiliar2;

    public MyClass() {
        //...
        painelPrincipal = new JPanel();
        painelAuxiliar1 = new JPanel();
        painelAuxiliar2 = new JPanel();
        getContentPane().add( painelPrincipal, BorderLayout.NORTH );
    }

    public void actionPerformed(ActionEvent e) {
        //if (e.getActionCommand().equals("Texto botao buttonAccao")) {
        if (e.getSource().equals(buttonAccao)) {
            JButton button1 = new JButton();
            //...
            painelAuxiliar1 = new JPanel();
            painelAuxiliar1.add(button1);
            //...
            painelPrincipal.remove(painelAuxiliar2);
            painelPrincipal.add(painelAuxiliar1);

            painelPrincipal.repaint();
            painelPrincipal.revalidate();
        } else {
            //...
        }
    }
}
```

Transição de painéis

Interfaces Gráficas – Contentores e Gestores de Posicionamento

```
public class MyClass extends JFrame implements ActionListener {
    JPanel painelPrincipal, painelAuxiliar1, painelAuxiliar2;

    public MyClass() {
        //...
        painelPrincipal = new JPanel();
        painelAuxiliar1 = new JPanel();
        painelAuxiliar2 = new JPanel();
        getContentPane().add( painelPrincipal, BorderLayout.NORTH );
    }

    public void actionPerformed(ActionEvent e) {
        //if (e.getActionCommand().equals("Texto botao buttonAccao")) {
        if (e.getSource().equals(buttonAccao)) {
            //...
        } else {
            JButton button2 = new JButton();
            //...
            painelAuxiliar2 = new JPanel();
            painelAuxiliar2.add(button2);
            //...
            painelPrincipal.remove(painelAuxiliar1);
            painelPrincipal.add(painelAuxiliar2);

            painelPrincipal.repaint();
            painelPrincipal.revalidate();
        }
    }
}
```

Transição de painéis

Interfaces Gráficas – Contentores e Gestores de Posicionamento

Ao invés de se reutilizar o mesmo painel, como sucede no exemplo anterior, o mais correcto é **criar novas classes de painéis (`JPanel`) ou janelas (`JFrame`) que serão trocadas (*swapped*)** a partir da classe principal.

A transição entre estas janelas é realizada através do método **`setVisible(boolean)`**, que torna visível (*true*) ou invisível (*false*) uma janela

Applets (JApplet) e Aplicações (JFrame, JWindow)

Já sabemos que as **JApplets** não podem funcionar **autonomamente**. Somente dentro de um *browser* ou de um *applet viewer*

O mesmo não se passa com as **aplicações**, que são **autónomas**

As **aplicações** devem ser derivadas da classe **JFrame**

Devem possuir método **main()** em vez de **init()** onde de deve criar a **janela** (**JFrame**) correspondente e iniciar a execução

Ao contrário de uma **JApplet**, uma **aplicação** pode conter um **menu**

Converter *applets* em aplicações

Interfaces Gráficas – Contentores e Gestores de Posicionamento

- (1) **Alterar o cabeçalho da classe**

herda de **JFrame** em vez de **JApplet**

- (2) **Mudar o nome do método `init()` para o nome da classe**, tornando-o assim no **construtor da classe** (eliminar *void* do cabeçalho)

- (3) Criar um **método `main()`**, inicializando o objecto com a interface:

```
public static void main (String[] args){  
    JFrame f = new NomeClasse();  
    f.setSize(300,300);  
    f.setVisible(true);  
}
```

- (4) **Adicionar um método `handleEvent` para tratar o fim da aplicação:**

```
public boolean handleEvent (Event e) {  
    if (e.id == Event.WINDOW_DESTROY)  
        System.exit(0);  
    return super.handleEvent(e);  
}
```

- Caso não seja tratado o encerramento da aplicação, a janela será removida do ecrã, porém o respectivo processo continuará activo e a consumir recursos

Encerramento de Janelas (JFrame)

Interfaces Gráficas – Contentores e Gestores de Posicionamento

O método `handleEvent` para tratar o fim da aplicação está definido como *deprecated*, não sendo assim recomendável o seu uso Futuro. Em alternativa existem 2 possíveis processos:

– (4a) **Associar uma acção por defeito** para o encerramento da janela e libertação dos recursos reservados, caso se **pretenda apenas encerrar a janela nem acções adicionais**:

```
setDefaultCloseOperation( JFrame.EXIT_ON_CLOSE );
```

– (4b) **Criar uma classe** `Terminador` que herda da **interface** `WindowAdapter` e reescreve o método `windowClosing(WindowEvent e)`, sendo o procedimento adequado caso se **pretenda realizar várias acções durante o encerramento** (e.g. encerrar o acesso a um SGBD ou libertar sockets)

```
class TerminadorJanela extends WindowAdapter {  
    public void windowClosing (WindowEvent e ){  
        System.exit(0);  
    }  
}
```

adicionando assim um **novo listener** à aplicação

```
addWindowListener( new TerminadorJanela() );
```

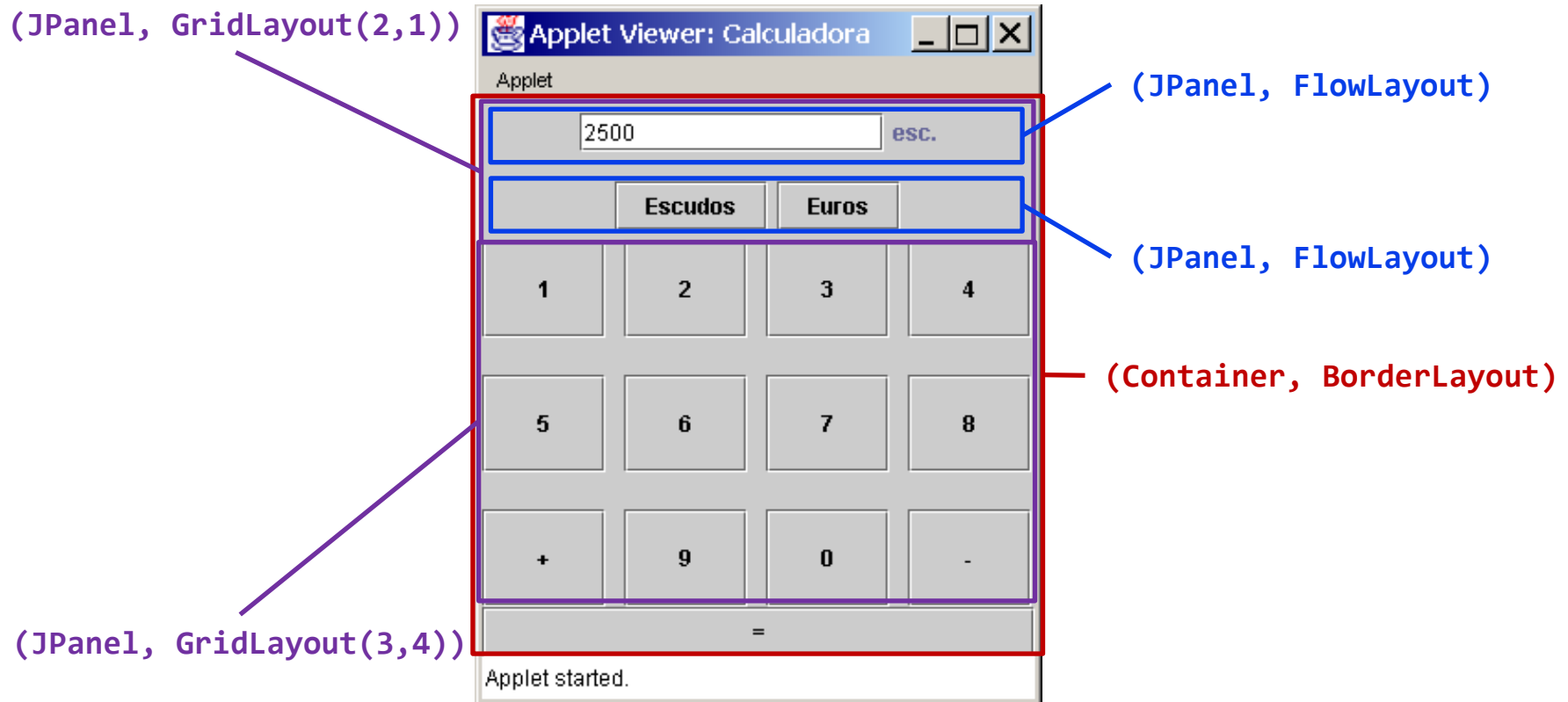
Se pretender **impedir o utilizador de encerrar a janela** basta usar o parâmetro `DO_NOTHING_ON_CLOSE`, também disponível na classe `WindowConstants`:

```
setDefaultCloseOperation( WindowConstants.DO_NOTHING_ON_CLOSE );
```



Aplicações

Interfaces Gráficas – Contentores e Gestores de Posicionamento



“Programação Orientada a Objectos”

António José Mendes

Departamento de Engenharia Informática, Universidade de Coimbra

“Java Swing”, 2ª Edição

James Elliott, Robert Eckstein (Editor), Marc Loy, David Wood, Brian Cole
O'Reilly, ISBN: 0596004087

“Thinking in Java, ”, 4ª Edição, Capítulo 14 *“Creating Windows & Applets”*

Bruce Eckel

Prentice Hall, ISBN: 0131872486

“The Java Tutorial – Creating a GUI with JFC/Swing”

Java Sun Microsystems

<http://java.sun.com/docs/books/tutorial/uiswing/index.html>

Referências

Interfaces Gráficos – Contentores e Gestores de Posicionamento

"The Definitive Guide to Java Swing", 3ª Edição

John Zukowski

Apress, ISBN: 1590594479

"Fundamentos de Programação em Java 2", Capítulo 12 *"Interfaces Gráficos"*

António José Mendes, Maria José Marcelino

FCA, ISBN: 9727224237

Revisto e adaptado a partir do original de António José Mendes e Maria José Marcelino